

1 RLHF方法脉络梳理

RLHF (Reinforcement Learning from Human Feedback) 的核心目标, 是在保留基座模型语言能力的同时, 利用人类偏好信号进一步约束模型输出行为, 使其在有用性、无害性、服从性以及推理质量等方面更符合预期。其经典流程通常包含监督微调 (SFT)、奖励建模 (Reward Modeling) 与策略优化 (Policy Optimization) 三个阶段。其中, 奖励模型首先根据成对偏好数据学习一个隐式奖励函数, 随后策略模型在奖励最大化与参考策略约束之间进行权衡优化。经典奖励模型训练目标可写为

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}_{\text{pref}}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))], \quad (1)$$

其中, x 表示提示, y_w 和 y_l 分别表示在人工偏好标注中胜出与落败的回答, $r_\phi(x, y)$ 为奖励模型输出。在此基础上, 经典 RLHF 的策略优化目标通常写为

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi(\cdot|x)} [r_\phi(x, y)] - \beta \mathbb{E}_{x \sim \mathcal{D}} [D_{\text{KL}}(\pi(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))], \quad (2)$$

其中 π_{ref} 为参考策略, β 用于控制偏离初始模型的程度。围绕这一基本框架, PPO、DPO、DRPO 与 DAPO 分别从策略更新稳定性、偏好优化简化、模型失配鲁棒性以及长链推理场景下的训练效率与稳定性等角度进行了改进。

2 PPO (Proximal Policy Optimization)

2.0.1 存在问题

- 传统策略梯度方法往往只能对每批样本进行一次更新, 样本利用率较低, 且梯度方差较大。
- 若直接进行大步长策略更新, 容易导致新旧策略分布偏移过快, 从而造成训练震荡甚至性能崩塌。
- TRPO 虽然能够通过信赖域约束提高稳定性, 但其二阶优化与约束求解过程实现复杂、计算代价较高, 不利于大规模工程落地。
- 在 RLHF 场景中, PPO 还需要同时维护奖励模型、价值网络以及参考策略, 训练链条较长, 调参成本较高。

2.1 近端策略优化算法

PPO 通过剪切概率比来限制策略更新幅度, 提高训练稳定性。其核心目标函数为:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

其中 $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$, ϵ 通常取 0.2。

算法流程如下:

1. 初始化策略网络参数 θ 、价值网络参数 ϕ , $\theta_{\text{old}} \leftarrow \theta$ 。
2. 循环迭代:

- (a) 用 $\pi_{\theta_{\text{old}}}$ 采集 N 条轨迹, 存储 (s, a, r) 。
- (b) 计算优势估计 $\hat{A}_t$ (如 GAE) 和回报 R_t 。
- (c) 对同一批数据执行 K 轮更新 (K 通常为 3 10):
 - i. 将数据划分为小批量。
 - ii. 对每个小批量, 计算 L^{CLIP} 、价值损失 $L^{\text{VF}} = (V_\phi(s_t) - R_t)^2$ 和熵奖励 S 。
 - iii. 总损失 $L^{\text{total}} = -L^{\text{CLIP}} + c_1 L^{\text{VF}} - c_2 S$ 。
 - iv. 梯度下降更新 θ 和 ϕ 。
- (d) $\theta_{\text{old}} \leftarrow \theta$ 。

伪代码实现:

Algorithm 1 PPO-Clip 算法

```

1: 初始化  $\theta, \phi, \theta_{\text{old}} \leftarrow \theta$ 
2: for 每次迭代 do
3:   采集轨迹  $\mathcal{D} \leftarrow \{(s_t, a_t, r_t)\}$  使用  $\pi_{\theta_{\text{old}}}$ 
4:   计算  $\hat{A}_t, R_t$  (GAE)
5:   for  $epoch = 1$  to  $K$  do
6:     for 每个小批量  $(s, a, \hat{A}, R)$  do
7:        $r_t(\theta) \leftarrow \pi_\theta(a|s)/\pi_{\theta_{\text{old}}}(a|s)$ 
8:        $L^{\text{CLIP}} \leftarrow \min(r(\theta)\hat{A}, \text{clip}(r(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A})$ 
9:        $L^{\text{VF}} \leftarrow (V_\phi(s) - R)^2$ 
10:       $S \leftarrow \mathcal{H}(\pi_\theta(\cdot|s))$ 
11:       $L^{\text{total}} \leftarrow -L^{\text{CLIP}} + c_1 L^{\text{VF}} - c_2 S$ 
12:      更新  $\theta \leftarrow \theta - \alpha \nabla_\theta L^{\text{total}}, \phi \leftarrow \phi - \alpha \nabla_\phi L^{\text{VF}}$ 
13:    end for
14:  end for
15:   $\theta_{\text{old}} \leftarrow \theta$ 
16: end for

```

3 DPO (Direct Preference Optimization)

3.0.1 存在问题

- PPO 式 RLHF 需要先训练奖励模型, 再进行在线强化学习优化, 训练流程较长且工程复杂。
- 奖励模型误差会通过强化学习阶段被进一步放大, 带来 reward hacking、优化不稳定和调参困难等问题。
- 在线采样与价值函数估计会引入较高计算开销, 不利于大模型高效对齐。

3.0.2 算法简介

DPO 的核心思想是: 在 KL 正则化的 RLHF 目标下, 最优策略与奖励函数之间存在闭式关系, 从而可以绕过显式强化学习步骤, 直接将偏好学习转化为一个二分类式目标。

其关键关系写为

$$r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x), \quad (3)$$

其中 $Z(x)$ 为与回答 y 无关的归一化项。因此，对于偏好对 (x, y_w, y_l) ，DPO 直接优化

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}_{\text{pref}}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]. \quad (4)$$

若将序列概率展开到 token 级别，则有

$$\log \pi_{\theta}(y|x) = \sum_{t=1}^{|y|} \log \pi_{\theta}(y_t|x, y_{<t}). \quad (5)$$

3.0.3 算法创新点

- 将带 KL 正则的 RLHF 目标改写为闭式偏好分类损失，省去了显式奖励模型学习后的在线 RL 更新过程。
- 训练时不再需要价值网络与复杂的 on-policy 采样流程，实现更简单、稳定且计算开销更低。
- 直接在“胜出回答应相对提高概率、落败回答应相对降低概率”的偏好层面进行优化，使目标与偏好数据更加一致。
- DPO 奠定了后续一系列“偏好优化替代 RLHF”的方法基础，成为 PPO 之外最具代表性的对齐路线之一。

4 GRPO (Group Relative Policy Optimization)

4.1 存在问题

- 在 PPO 中，通常需要额外训练一个价值模型 V_{ψ} 来估计优势函数。对于大语言模型而言，价值模型的规模往往与策略模型同量级，这会显著增加训练的显存占用与计算开销。
- 大语言模型强化学习任务中的奖励通常主要作用于完整回答或最终结果，而不是天然具备逐 token 的稠密监督信号，因此价值函数的学习往往较为困难，误差也容易传递到策略更新过程中。
- 对于同一个问题，不同回答之间天然存在显著的相对优劣关系。传统 PPO 更多依赖绝对奖励与价值基线来构造优势，未能充分利用“问题多答”这一组内相对比较信息。

4.2 算法简介

GRPO 的核心思想是：对于每个问题 q ，从旧策略 π_{old} 中采样一组回答 $\{o_i\}_{i=1}^G$ ，不再单独训练价值模型，而是直接利用这组回答的相对奖励构造优势函数，再以类似 PPO 的裁剪目标更新当前策略 π_{θ} 。

其目标函数可写为

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E} [q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)] \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left(\min(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t}) - \beta L \right) \right] \quad (6)$$

其中,

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})} \quad (7)$$

表示当前策略与旧策略在第 i 个回答、第 t 个 token 处的概率比。

与 PPO 的关键区别在于, GRPO 不再利用价值模型估计优势, 而是直接根据同一组回答的奖励构造组相对优势。若第 i 个回答的奖励为 r_i , 则其优势函数可写为

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}, \quad \mathbf{r} = (r_1, r_2, \dots, r_G). \quad (8)$$

这意味着: 同一问题下, 各回答的平均奖励被当作基线, 标准差用于归一化, 从而使模型学习“这一组回答中谁更好、好多少”。

在 KL 正则项的具体实现中, GRPO 常采用如下形式的 token-level KL 估计 (k3估计器), 而不是最常规的KL散度:

$$D_{\text{KL}}(\pi_{\theta} \| \pi_{\text{ref}}) = \mathbb{E}_{o \sim \pi_{\theta}} \left[\log \frac{\pi_{\theta}(o)}{\pi_{\text{ref}}(o)} \right]. \quad (9)$$

其中, o 表示从当前策略 π_{θ} 生成的输出, π_{ref} 表示参考策略。该式是 KL 散度的标准定义, 含义是: 在当前策略生成的输出分布下, 衡量当前策略与参考策略之间的整体差异。

GRPO 常采用如下更稳定的逐 token 近似形式:

$$D_{\text{KL}}[\pi_{\theta} \| \pi_{\text{ref}}] \approx \frac{\pi_{\text{ref}}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta}(o_{i,t} | q, o_{i,<t})} - \log \frac{\pi_{\text{ref}}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta}(o_{i,t} | q, o_{i,<t})} - 1. \quad (10)$$

令

$$x = \frac{\pi_{\text{ref}}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}, \quad (11)$$

则上式可写为

$$k_3(x) = x - \log x - 1. \quad (12)$$

该形式的优点在于: 当 $x = 1$ 时, 即当前策略与参考策略在该 token 上概率一致时, 有 $k_3(1) = 0$; 当二者偏离时, 该值始终非负并随偏离增大而增大, 因此可作为一个更平滑、更稳定的 KL 正则近似项。(无偏、低房产证)

因此, GRPO 的整体流程可以概括为: 先对同一问题采样多个回答, 再由奖励模型或规则函数为每个回答打分, 随后在组内计算相对优势, 最后通过带裁剪的策略目标和 KL 正则共同更新当前策略。

4.3 算法创新点

- **去除价值模型。** GRPO 最显著的创新是去掉了 PPO 中与策略模型并行训练的价值模型, 直接用组内奖励均值构造基线, 从而显著降低训练时的显存与计算资源开销。

- **利用组内相对比较信息。** GRPO 通过“问题多答”的方式，将不同回答之间的相对质量差异直接转化为优势信号，使算法天然适配偏好式奖励与比较式奖励场景。
- **保留 PPO 的稳定更新机制。** 虽然去掉了价值模型，GRPO 仍然保留了 PPO 中基于重要性采样比率的裁剪目标，因此能够在降低资源消耗的同时维持训练稳定性。
- **更适合大语言模型强化学习。** 在大语言模型场景中，奖励往往更适合落在整条回答层面而非逐 token 层面。GRPO 通过组相对奖励替代逐时刻价值估计，使其在数学推理、逻辑推理和可验证任务中更自然，也更容易工程实现。

5 DeepSeek-R1-Zero

5.1 存在问题

- 传统大语言模型在推理任务上的后训练通常依赖监督微调（SFT）作为冷启动步骤。虽然这种方式能够改善回答风格、可读性与指令跟随能力，但也意味着模型的推理模式在一开始就被人工示范数据所显式约束，因而难以回答一个更基础的问题：如果不给模型提供任何推理示范，仅依靠强化学习和可验证奖励，模型能否自行发展出较强的推理能力。
- 在数学、逻辑和编程等可验证任务中，若仍沿用基于奖励模型的传统 RLHF 路线，则奖励质量容易受到奖励模型误差、偏差与可利用漏洞的影响，进而出现 reward hacking 等问题。因此，对于这类“答案可自动检验”的任务，更自然的思路是直接采用规则型奖励，而非额外训练一个神经奖励模型。
- 即使底层已经有 GRPO 这样的强化学习算法，算法本身也只解决“如何更新策略参数”的问题，并不能自动回答“训练从何起步、奖励如何定义、输出格式如何约束、训练后会出现什么行为特征”等更完整的系统性问题。DeepSeek-R1-Zero 正是在这一背景下提出的具体训练方案。

5.2 算法简介

DeepSeek-R1-Zero 的核心思想是：以 DeepSeek-V3-Base 作为起点，不经过监督微调冷启动，直接在可验证推理任务上施加大规模强化学习，从而探索大语言模型是否能够通过纯 RL 过程自发形成推理能力。

在优化算法层面，DeepSeek-R1-Zero 采用 GRPO 作为强化学习框架。对于给定问题 q ，先由旧策略采样一组回答 $\{o_i\}_{i=1}^G$ ，再根据组内奖励的相对大小构造优势项，并用裁剪后的策略目标更新当前模型参数。其目标函数可写为

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|O_i|} \sum_{t=1}^{|O_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t} \right) - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right], \quad (13)$$

其中

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\text{old}}(o_{i,t} \mid q, o_{i,<t})}, \quad (14)$$

组相对优势写为

$$\hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^G)}{\text{std}(\{R_j\}_{j=1}^G)}. \quad (15)$$

在奖励设计上, DeepSeek-R1-Zero 不使用神经奖励模型, 而是直接采用基于规则的奖励机制。其总奖励可以概括为

$$R = R_{\text{acc}} + R_{\text{format}}, \quad (16)$$

其中, R_{acc} 表示准确性奖励, 用于判断模型最终答案是否正确; R_{format} 表示格式奖励, 用于要求模型按预定结构输出推理过程与最终答案。

在输出模板上, DeepSeek-R1-Zero 要求模型显式区分“思维过程”和“最终答案”, 可写为

$$\langle \text{think} \rangle \text{ reasoning process } \langle / \text{think} \rangle \langle \text{answer} \rangle \text{ answer } \langle / \text{answer} \rangle. \quad (17)$$

这种设计一方面有助于利用格式奖励约束模型输出, 另一方面也便于观察模型在强化学习过程中是否逐渐出现更长链、更显式的推理行为。

5.3 算法创新点

- **纯强化学习冷启动。** DeepSeek-R1-Zero 最重要的创新在于, 它不依赖 SFT 作为先验引导, 而是直接在基座模型上进行大规模强化学习, 从而验证了“推理能力可以通过纯 RL 过程被激发”这一点。这使得研究者能够更直接地观察模型推理能力的自发演化过程。
- **规则型奖励替代奖励模型。** 在可验证任务上, DeepSeek-R1-Zero 采用准确性奖励与格式奖励构成的规则奖励系统, 避免了训练独立奖励模型所带来的额外复杂度与潜在误差, 也使训练过程更适合大规模开展。
- **推理行为的自发涌现。** 论文表明, 随着 RL 训练推进, 模型会逐渐表现出更长的回答长度、更明显的自我反思、重新审视中间步骤以及尝试不同解法等行为模式。这说明强化学习不仅提高了最终答案正确率, 也改变了模型外显的推理方式。
- **揭示纯 RL 路线的边界。** DeepSeek-R1-Zero 虽然展示了纯 RL 激发推理能力的潜力, 但同时也暴露出若干局限, 如回答可读性较差、语言混杂等问题。这些现象表明, 纯 RL 能够有效提升推理能力, 但未必足以同时保证通用对话质量和良好的输出风格, 这也为后续 DeepSeek-R1 采用多阶段训练提供了动机。

5.4 和GRPO的关系

DeepSeek-R1-Zero 与 GRPO 不是两个并列的方法, 而是“具体训练方案”与“底层优化算法”之间的关系。

GRPO 是一种强化学习优化算法, 其核心作用是: 在不训练独立价值模型的前提下, 通过组内样本相对奖励来估计优势, 并据此更新策略参数。因此, GRPO 回答的是“参数应该如何更新”的问题。

而 DeepSeek-R1-Zero 则是在 GRPO 基础上的完整实例化方案。它进一步补充了以下几个关键要素: 第一, 指定基座模型为 DeepSeek-V3-Base; 第二, 规定训练起点为“无 SFT 冷启动”;

第三，规定任务类型主要为可验证推理任务；第四，规定奖励形式为“准确性奖励 + 格式奖励”；第五，规定模型输出需采用 `<think>` 与 `<answer>` 的结构化模板。

因此，若用一句话概括二者关系，可以写为：

GRPO 是 DeepSeek-R1-Zero 所采用的强化学习优化算法，而 DeepSeek-R1-Zero 是“基座模型 + 纯 RL 冷启动 + 规则奖励 + 输出模板 + GRPO 优化”共同构成的完整训练方案。

换言之，GRPO 决定的是“怎么优化”，DeepSeek-R1-Zero 决定的是“在什么模型、什么任务、什么奖励和什么训练设定下使用这种优化”。

6 DAPO (Decoupled Clip and Dynamic sAmpling Policy Optimization)

6.0.1 存在问题

- 直接将 PPO 或 GRPO 用于长链推理任务时，容易出现策略熵快速下降、采样多样性不足以及 entropy collapse 问题。
- 当同一 prompt 采样得到的整个 group 全对或全错时，组内奖励方差接近于零，优势信号退化，导致训练效率下降。
- 长链推理场景下，若仍采用 sample-level 的平均损失，长回答与短回答之间会产生不合理的梯度加权偏差。
- 超长输出被截断后若仍按原始规则奖励处理，会把“答案错误”与“长度超限”混杂在一起，从而引入额外奖励噪声。

6.0.2 算法简介

DAPO 建立在 GRPO 的思路之上，但对裁剪策略、采样机制、损失粒度和超长惩罚进行了系统改进。其目标函数写为

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) \hat{A}_{i,t} \right) \right], \quad (18)$$

并要求该 group 同时包含正确与错误样本，即

$$0 < |\{o_i \mid \text{is_equivalent}(a, o_i)\}| < G. \quad (19)$$

其中，重要性采样比率为

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}, \quad (20)$$

组相对优势写为

$$\hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^G)}{\text{std}(\{R_i\}_{i=1}^G)}. \quad (21)$$

与标准 GRPO 不同，DAPO 采用解耦裁剪，即对上界与下界分别使用 ϵ_{high} 与 ϵ_{low} ，从而放宽正优势样本向上更新的空间；同时通过动态采样仅保留具有有效梯度信号的 group，缓解全对或全错带来的无效更新问题。此外，DAPO 还使用 token-level 的归一化方式，即以总 token 数而非样本数进行归一化，以更适配长 CoT 场景。

在奖励构造上，其可验证任务采用规则奖励

$$R(\hat{y}, y) = \begin{cases} 1, & \text{is_equivalent}(\hat{y}, y), \\ -1, & \text{otherwise,} \end{cases} \quad (22)$$

并进一步加入柔性超长惩罚

$$R_{\text{length}}(y) = \begin{cases} 0, & |y| \leq L_{\text{max}} - L_{\text{cache}}, \\ \frac{(L_{\text{max}} - L_{\text{cache}}) - |y|}{L_{\text{cache}}}, & L_{\text{max}} - L_{\text{cache}} < |y| \leq L_{\text{max}}, \\ -1, & L_{\text{max}} < |y|. \end{cases} \quad (23)$$

6.0.3 算法创新点

- 提出 Clip-Higher 机制，通过增大 ϵ_{high} 而保留 ϵ_{low} ，缓解高概率 token 持续垄断、低概率但潜在有效 token 难以上升的问题，从而抑制熵塌缩。
- 提出 Dynamic Sampling，仅保留同时含有成功与失败样本的 group，使组相对优势始终具有有效区分度，提高训练效率与稳定性。
- 将 sample-level loss 改为 token-level loss，以总 token 数归一化，更适合长链推理场景下不同长度样本的公平优化。
- 通过 Soft Overlong Punishment 对超长截断样本施加柔性惩罚，将“长度过长”与“答案错误”区分开来，从而减少奖励噪声。
- DAPO 的贡献不仅是对 GRPO 的小修补，而是面向长 CoT 推理场景对裁剪、采样、损失归一化与奖励塑形四个关键环节的联合重构。

7 大语言模型强化学习中的系统异构性

7.1 推理架构与训练架构的异构性

首先，大语言模型强化学习同时包含生成式推理与梯度训练两类本质不同的计算过程。对于 π_{old} 的回答采样而言，系统需要执行自回归生成，即在给定提示 q 后逐 token 地扩展回答序列；该过程强依赖 KV cache，生成长度动态变化，并且更强调低时延与高并发的推理服务能力。相比之下，策略模型 π_{θ} 与价值模型 V_{ψ} 的训练阶段则更偏向规则的大批量张量计算，其目标是提升 GPU 吞吐率、增大 batch 并提高反向传播效率。因此，最适合推理的系统架构并不等同于最适合训练的系统架构：前者偏向流式生成与请求调度，后者偏向数据并行、梯度同步与高吞吐矩阵运算。这种推理端与训练端在执行模式上的差异，构成了大语言模型强化学习的第一层异构性。

7.2 推理时间与训练时间的异构性

其次，采样、打分与训练在时间尺度上也并不一致。回答生成往往是整个流程中最慢的阶段之一，因为模型必须逐 token 地进行自回归解码；在此基础上，参考模型打分、奖励模型推理以及价值估计还会进一步引入额外时延。相较而言，一旦样本收集完成，训练阶段的前向与反向传播通常可以以较高吞吐率集中执行。这就导致样本生成速度与样本消耗速度并不匹配，从而容易产生参数版本滞后问题。形式上，训练时真正被更新的是当前策略 π_θ ，而用于构造训练样本的往往是较早时刻的旧策略 π_{old} ，即

$$\pi_\theta \neq \pi_{\text{old}}. \quad (24)$$

当这一差异持续扩大时，就会产生显著的 *policy lag* 现象：训练端已经基于更“新”的模型参数执行了若干轮更新，而推理端仍在使用较“旧”的策略持续采样，从而使优化目标与真实数据分布之间逐渐出现偏移。对于依赖重要性采样比率的 PPO / GRPO 类方法而言，这种时序异构性会直接影响比率项

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}, \quad (25)$$

使得训练稳定性与理论近似条件同时受到挑战。

7.3 采样样本之间的异构性

再次，即使来自同一提示 q 的一组采样结果 $\{o_i\}_{i=1}^G$ ，不同样本之间仍然存在显著差异。这种差异首先体现在长度层面：部分回答较短，可迅速完成生成与打分；另一些回答则可能包含长链推理过程，生成时间、显存占用与奖励评估代价都明显更高。其次，不同样本的奖励结构也并不一致，有的样本可能在格式上正确但最终答案错误，有的样本则可能在中间推理阶段表现较好而在最后一步失败，从而导致样本间的有效梯度信号强弱不同。再者，在可验证任务中，同一组采样结果还可能出现“全对”或“全错”的情况，使组内奖励方差显著降低，进而削弱相对优势信号。因此，大语言模型强化学习中的 batch 并不是由一组同质样本构成，而更接近于由不同长度、不同难度、不同奖励形态的序列共同组成的异构集合。

7.4 方案一：基于旧策略样本复用的批量训练

该方案的基本思路是：先使用旧策略 π_{old} 采样一批较大规模的数据，再将这批数据划分为多个 minibatch，供训练端连续进行多轮参数更新。从直观上看，它属于“先集中采样，再反复利用旧样本训练”的批处理模式。

这种方案的优点在于实现简单，容易兼容现有 PPO / GRPO 训练框架，并且能够通过重复利用同一批样本提高训练吞吐率和样本利用率。由于训练阶段不必每更新一步都重新等待推理端生成新数据，因此整体 GPU 利用率通常较高，工程改造成本也相对较低。

其缺点在于，随着训练端连续更新，当前策略 π_θ 会逐渐偏离最初采样所使用的旧策略 π_{old} ，从而产生明显的版本差异。这种策略滞后会使后续梯度估计与真实当前策略分布之间出现偏移，进而影响训练稳定性与优化效果。因此，该方案虽然高效，但本质上是以“样本新鲜度下降”为代价换取“实现简单和吞吐提升”。

7.5 方案二：PipelineRL式边采样边训练

该方案的基本思路是：推理端持续采样，训练端持续消费新样本并更新参数，同时在模型更新后尽快将新权重同步回推理端。从直观上看，它属于“边生成、边训练、边同步新模型”的流式管线模式。

这种方案的优点在于，它能够显著减小旧策略与当前策略之间的时间差，从而缓解策略滞后问题，使新生成的数据更接近当前训练中的模型分布。同时，由于采样与训练可以并发执行，系统整体资源利用率通常更高，尤其适合长序列生成场景下的大模型强化学习。

其缺点在于系统实现更复杂，需要处理推理端与训练端之间的在线调度、权重同步以及缓存一致性问题。即使新权重已经同步到推理端，先前生成过程中留下的 KV cache 仍可能对应旧版本参数，因此版本差异并不会被完全消除。换言之，该方案能够显著缓解而非彻底消除异构带来的时滞问题，但其工程复杂度明显高于第一种方案。